

Celebal Excellence Internship

[Week 6 Task – Basics of Pyspark]

Title: CEI – week 6 task submission

Name: Anushka Manoj Patil

College name: Dr. D. Y. Patil Pimpri, Pune

Q1: Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Driver

- The Driver is the main program.
- It creates the SparkSession.
- Converts the code into tasks.
- Builds the DAG (Directed Acyclic Graph).
- Sends tasks to executors.
- Collects the final results.

Cluster Manager

- Allocates CPU and memory resources.
- Starts Executor processes.
- Examples:
 - Standalone Cluster Manager
 - YARN
 - Kubernetes
 - Mesos

Executors

- Worker processes that execute tasks.
- Store data in memory.
- Perform computations.
- Return results to the Driver.

Flow

Driver



Cluster Manager



Executors

Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Spark does not execute transformations immediately.

Instead, it:

- Records all transformations.
- Builds a DAG (Lineage Graph).
- Waits until an Action (show(), count(), collect()) is called.
- Optimizes the execution plan before running.

Benefits

- Eliminates unnecessary computations.
- Combines multiple transformations.
- Reduces disk I/O.
- Improves execution speed.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

```
df = spark.read.csv  
(  
    "data/source.csv",  
    header=True,  
    inferSchema=True  
)
```

Q4: What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

CSV	Parquet
Row-based storage	Columnar storage
Larger file size	Smaller compressed size
Slower	Faster
No schema stored	Schema stored
Reads entire rows	Reads only required columns

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

```
df.filter(col("category") == "Electronics") \
    .select("product_id", "price") \
    .show()
```

Q6: Write the code to "revise" a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

Query:

```
from pyspark.sql.functions import col
```

```
df = df.withColumnRenamed("old_name", "new_name")
```

```
df = df.withColumn(  
    "price",  
    col("price").cast("double")  
)
```

Q7: How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Spark stores all transformations in a Lineage Graph (DAG).

If an Executor fails:

- Spark does not reload the entire dataset.
- It recomputes only the lost partition using the DAG.
- This provides automatic fault tolerance.

Advantages:

- No data replication required.
- Efficient recovery.
- Faster fault handling.

Q8: Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

Query:

```
df_orders.filter(  
    (col("status") == "Completed") &  
    (col("amount") > 1000)  
)
```

Q9: Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Answer:

Predicate Pushdown means Spark sends filter conditions directly to the Parquet file.

Example:

```
df.filter(col("age") > 30)
```

Instead of reading the entire file:

- Spark reads only matching row groups.
- Less data is loaded into memory.

Benefits:

- Faster execution
- Less memory usage
- Reduced disk reads

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

```
from pyspark.sql.functions import col
```

```
df = df.withColumn(  
    "final_price",  
    col("base_price") * 1.18  
)
```

Q11: What is the difference between Transformations and Actions? Provide two examples of each.

Transformations

Lazy

Return a new DataFrame

Actions

Trigger execution

Return result/output

Transformations

- `filter()`
- `select()`

Example:

```
df.filter(col("salary") > 50000)
```

```
df.select("name", "salary")
```

Actions

- `show()`
- `count()`

Example:

```
df.show()
```

```
df.count()
```

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

Query:

```
df = spark.read.parquet("path/to/input")

df.filter(col("user_id").isNotNull()) \
    .write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("path/to/output")
df.show()
```

Q13: In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Client Mode

Driver runs on local machine

User machine must stay connected

Best for development

Cluster Mode

Driver runs inside the cluster

User can disconnect after submission

Best for production

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

```
df.filter(  
    (col("region") == "North") |  
    (col("priority") == "High")  
).show()
```

Q15: When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

Answer

.show(5)

- Displays only the first 5 rows.
- Uses very little memory.
- Safe for large datasets.
- Good for data exploration.

.collect()

- Brings the entire dataset to the Driver.
- Can consume huge memory.
- May cause OutOfMemoryError on large datasets.
- Should be avoided for very large data.